

Loops in Java

- Looping in programming languages is a feature which facilitates the execution of a set of instructions/functions repeatedly while some condition evaluates to true.
- Java provides three ways for executing the loops. While all the ways provide similar basic functionality, they differ in their syntax and condition checking time.

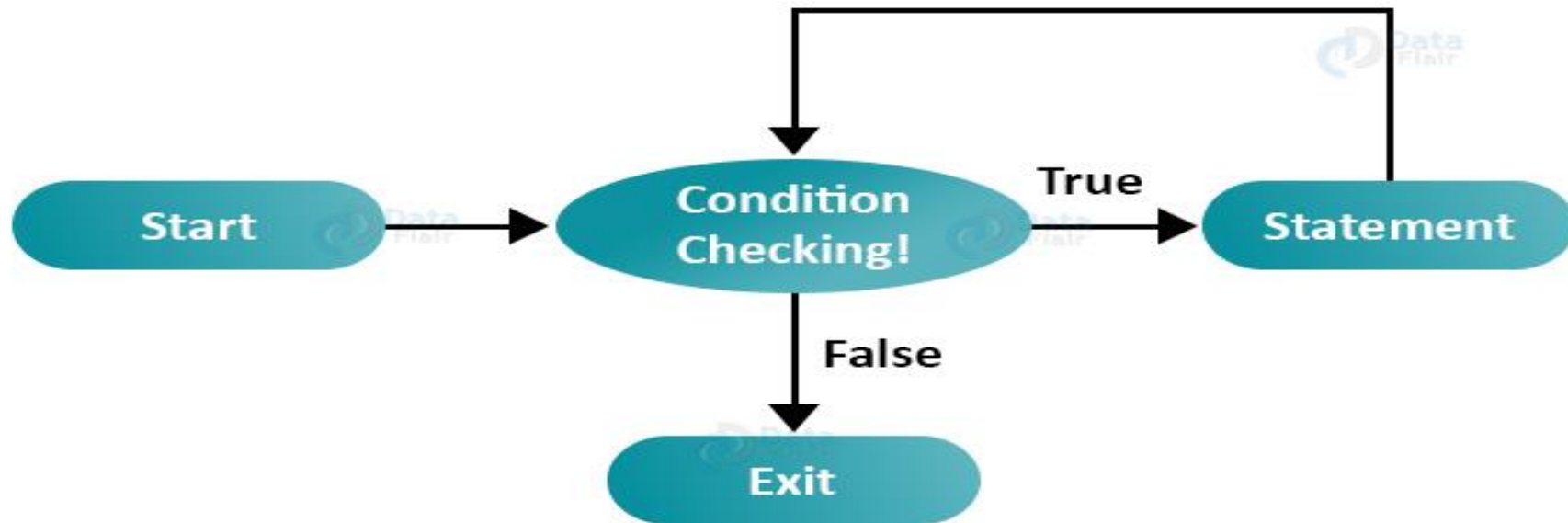
1. While Loop

- A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition.
- A while loop iterates through a set of statements till its boolean condition returns false. As long as the condition given evaluates to true, the loop iterates.
- The condition of the loop structure is checked at first and then the control proceeds into the loop structure only if the condition evaluates to true. Hence it is called an entry-controlled loop.
- The body of the loop generally contains a variable which controls the boolean condition mentioned.

The basic syntax of Java while loop is:

- while(boolean condition)
{
 //statements;
}

Flow Diagram for Whileloop



2. For Loop

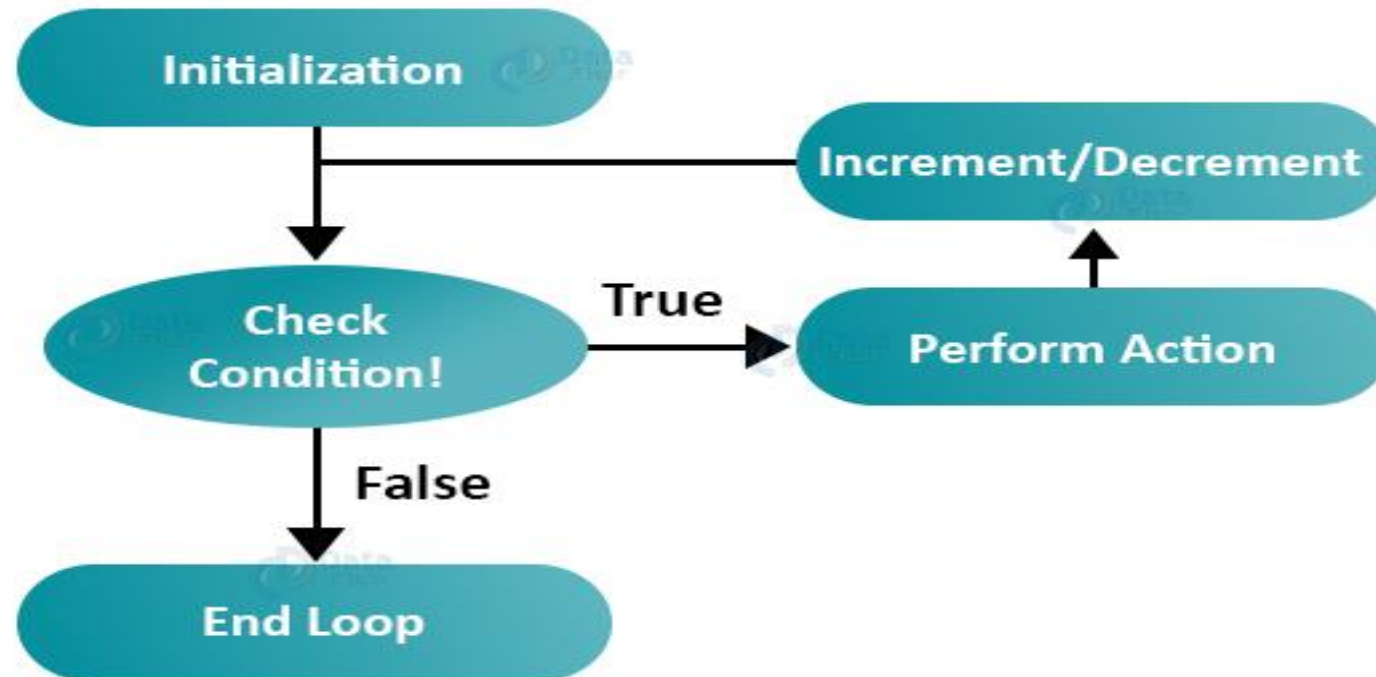
- for loop provides a concise way of writing the loop structure. Unlike a while loop, a for statement consumes the initialization, condition and increment/decrement in one line thereby providing a shorter, easy to debug structure of looping.
- The initializing statement marks the beginning of the loop structure. It contains a variable with some initial value that is defined by the programmer. This is the value of the control variable when the control shifts into the loop. However this statement is executed only once.

- **Initialization condition:** Here, we initialize the variable in use. It marks the start of a for loop. An already declared variable can be used or a variable can be declared, local to loop only.
- **Testing Condition:** It is used for testing the exit condition for a loop. It must return a boolean value. It is also an **Entry Control Loop** as the condition is checked prior to the execution of the loop statements.
- **Statement execution:** Once the condition is evaluated to true, the statements in the loop body are executed.
- **Increment/ Decrement:** It is used for updating the variable for next iteration.
- **Loop termination:** When the condition becomes false, the loop terminates marking the end of its life cycle.

The basic syntax of java for loop goes like this:

- `for(initializing statement;testing condition;increment/decrement)`
 {
 //code to be iterated
 }

Flow Diagram for Forloop



a. Enhanced for Loop

- Java also includes another version of for loop introduced in Java.
- This is similar to a for loop except that it has some enhanced features. It can be used to iterate over the elements of a collection without knowing the index of each element. You also need not know the size of the collection.
- However there are some limitations to this enhanced for loop. This loop object is immutable which means the values which are retrieved during the execution of the loop are read only. You cannot change the values in the collection while using this loop which is easily done by other loops.

The syntax for Java enhanced for loop is:

- `for(<datatype> <variable_name>:<collection_name>)`
 {
 //Statements;
 }

3. Do While Loop

- do while loop is similar to while loop with only difference that it checks for condition after executing the statements, and therefore is an example of **Exit Control Loop**.
- This is an exit-controlled loop because of the fact that it checks the condition after the statements inside it are executed.
- The Java *do-while loop* is executed at least once because condition is checked after loop body.

Syntax:

- do{
 //code to be executed / loop body
 //update statement
}while (condition);

Flow Diagram for Dowhileloop

